MedCompress: Compressing Medical Imaging Models for Endpoint Deployment on Mac and Windows

Abhishek Shekhar
abhishek.shekhar@medcompress.dev
https://github.com/Abhi183/medcompress

Abstract

Deep learning models for medical image analysis assume GPU hardware that clinic workstations, field laptops, and telemedicine terminals do not have. I present MedCompress, an open-source benchmark that tests how far three compression strategies can push medical imaging models toward CPU-only inference on macOS and Windows. The strategies are quantization-aware training (QAT), knowledge distillation (KD), and a sparse attention mechanism I adapted from Memory Sparse Attention [Chen et al., 2026]. I evaluate on two clinical tasks: melanoma classification with the ISIC 2020 dataset (33,126 dermoscopy images) and brain tumor segmentation with BraTS 2021 (multimodal MRI). QAT at INT8 cuts model size 3.8× with 1.4% AUC loss on melanoma detection. Distillation from a full U-Net teacher to a lightweight student keeps 94.3% of teacher Dice on brain tumor segmentation at 7.9× fewer parameters. Sparse attention with kernel-4 pooling and top-8 routing reduces attention memory 24.5× at 0.7% AUC cost. The combined KD + QAT INT8 pipeline yields a 2.7 MB melanoma classifier running in 8.3 ms and a 4.1 MB segmentation model running in 12.6 ms, both on CPU. Code, export pipelines, and a desktop application are released at `https://github.com/Abhi183/medcompress`.

1 Introduction

I manage fleets of Mac and Windows machines for a living. I know what ships through Jamf and SCCM and what does not. A 124 MB segmentation model with GPU dependencies does not. A 4 MB TFLite file does.

Medical imaging models are getting more accurate every year [Gholami et al., 2022]. The hardware at the point of care has not kept pace. Clinics run five-year-old iMacs. Field offices run Windows laptops with integrated graphics. Telemedicine terminals run whatever procurement found cheapest. Cloud inference addresses the compute gap but creates others: latency on spotty connections, data governance headaches when patient images leave the machine, and vendor lock-in that IT departments have learned to avoid the hard way.

I wanted to find out how far I could compress a medical imaging model before it stops being clinically useful, and whether the result can be deployed the same way I deploy any other endpoint software—pushed through MDM, versioned alongside the app, rolled back with a file swap.

MedCompress is the benchmark I built to answer that. It implements quantization-aware training, knowledge distillation, and sparse attention compression, evaluates each on melanoma classification and brain tumor segmentation, and exports compressed models to TFLite and ONNX for CPU inference. The sparse attention module borrows KV cache pooling and top-k routing from Memory Sparse Attention [Chen et al., 2026], a technique originally designed for 100M-token language contexts. I adapted it for Vision Transformer attention in dermoscopy, where it turned out to work surprisingly well (Section 4.4).

I also built a desktop application that wraps the compressed models in a GUI. A clinician opens an image, clicks a button, and gets a classification or segmentation result, no cloud, no GPU, no Python installation required.

2 Related Work

2.1 Model Compression

Quantization reduces weight precision from 32-bit floating point to 8-bit integers or lower [Jacob et al., 2018, Gholami et al., 2022]. Post-training quantization is simple but tends to damage accuracy on medical images where the signal lives in subtle color gradients. QAT simulates quantization during training so the network learns to tolerate the noise [TensorFlow Team, 2023a]. Jacob et al. [Jacob et al., 2018] reported near-

lossless INT8 inference on ImageNet, though medical tasks with extreme class imbalance and fine spatial features fail differently.

Knowledge distillation trains a compact student network to match softened probability outputs from a larger teacher [Hinton et al., 2015]. The temperature parameter controls how much of the teacher's internal ranking over classes leaks through the soft targets. Feature-level distillation, matching intermediate layer representations through MSE loss, outperforms logit-only distillation on segmentation where spatial structure matters at every pixel [Romero et al., 2015].

Pruning [Han et al., 2015] and neural architecture search are complementary approaches I did not evaluate here. Both could be stacked on top of QAT and KD, and I plan to investigate that combination in follow-up work. Choudhary et al. [Choudhary et al., 2020] provide a broad survey of compression methods for reference.

2.2 Efficient Attention

Standard self-attention is quadratic in sequence length. For a Vision Transformer [Dosovitskiy et al., 2021] processing a 224×224 image at patch size 16, that is 196 tokens attending to each other at every layer. Linformer [Wang et al., 2020] projects to lower dimensions. Performer [Choromanski et al., 2021] uses random feature approximations. FlashAttention [Dao et al., 2022] keeps exact attention but restructures memory access patterns.

Chen et al. [Chen et al., 2026] took a different route with Memory Sparse Attention (MSA). They compress the KV cache by averaging contiguous token chunks (chunk-mean pooling) and route each query to only the top-k most relevant compressed chunks. On language benchmarks, MSA scaled to 100 million tokens with under 9% degradation. The 94.84% accuracy on needle-in-a-haystack retrieval at 1M tokens is what originally caught my eye—it suggested the mechanism could handle spatial redundancy in images too.

2.3 Medical Imaging on Consumer Hardware

Prior work on compressing medical models has targeted mobile phones [Pacheco et al., 2024] and edge devices [Qayyum et al., 2023]. The endpoint scenario differs: workstations have 8–32 GB of RAM and multi-core CPUs but no GPUs. The binding constraint shifts from raw model size to CPU inference latency. A 10 MB model is fine. 300 ms per image is not.

TensorFlow Lite [TensorFlow Team, 2023b] and ONNX Runtime [ONNX Runtime Team, 2023] both handle CPU inference on macOS, Windows, and Linux. They are production-ready. What nobody had done was evaluate compressed medical models on these runtimes with endpoint deployment constraints—model size under 10 MB, latency under 50 ms, RAM under 100 MB.

3 Methods

3.1 Datasets

ISIC 2020. 33,126 dermoscopy images for binary melanoma classification [Codella et al., 2019]. Melanoma prevalence sits around 2%, so the dataset is heavily lopsided. I resized images to 224×224, normalized to $[-1, 1]$, and used inverse-frequency class weighting. Augmentation included random flips, brightness (±0.2), and contrast (±0.2). I split 70/15/15 for train/validation/test with stratification on the target label.

BraTS 2021. Multi-modal brain MRI volumes (T1, T1ce, T2, FLAIR) with four segmentation classes: background, necrotic core, peritumoral edema, enhancing tumor [Baid et al., 2021]. I remapped labels from $\{0, 1, 2, 4\}$ to $\{0, 1, 2, 3\}$ for standard one-hot encoding and used a 2.5D approach: 3 adjacent axial slices stacked across 4 modalities produce a 12-channel 128×128 input. This gives some volumetric context without the 3D convolutions that would make CPU inference impractical. I filtered background-only slices and z-score normalized each volume independently.

3.2 Baseline Models

For classification I used EfficientNetB0 [Tan and Le, 2019] pretrained on ImageNet with a custom head: global average pooling, batch normalization, dropout (0.3), a 256-unit dense layer with ReLU, dropout (0.15), and a sigmoid output. I unfroze the last 20 backbone layers for domain adaptation and trained with binary cross-entropy, Adam at 10^{-4}, and early stopping on validation AUC (patience 7 epochs).

For segmentation I implemented two U-Net [Ronneberger et al., 2015] variants. The full model has four encoder stages with [64, 128, 256, 512, 1024] filters and is the teacher. The lite model has three stages with [32, 64, 128, 256] filters—about 8× fewer parameters—and is the student. Both use combined Dice + cross-entropy loss, Adam at 10^{-3}, with early stopping on validation Dice.

3.3 Quantization-Aware Training

I used the TensorFlow Model Optimization Toolkit [TensorFlow Team, 2023a] to wrap trained baselines with fake-quantization nodes that simulate INT8 arithmetic during forward passes. Fine-tuning ran 10 epochs at 10^{-5} learning rate. After that I stripped the QAT wrappers and exported to TFLite with full INT8 quantization. The INT8 conversion needs a representative calibration dataset; I used 200 randomly sampled training images. I also exported FP16 variants for comparison.

3.4 Knowledge Distillation

For classification the teacher was EfficientNetB3 (12M parameters) and the student was MobileNetV3-Small [Howard et al., 2019] (2.5M parameters). For segmentation the teacher was the full U-Net and the student was the lite U-Net.

The distillation loss combines a soft-target KL divergence with a hard-target cross-entropy:

$$\mathcal{L} = \alpha \cdot \mathrm{KL}\Big(\sigma\Big(\frac{z_t}{T}\Big)\Big\|\sigma\Big(\frac{z_s}{T}\Big)\Big) + (1-\alpha)\cdot \mathrm{CE}(y, z_s) \quad (1)$$

where z_t and z_s are teacher and student logits, T is temperature, α weights the two terms, and σ is softmax (sigmoid for binary classification). I used T=4.0, α=0.7 for classification and T=3.0, α=0.6 for segmentation. For segmentation I added feature-level MSE distillation with 1×1 conv adapters for dimension matching, following Romero et al. [2015].

3.5 Sparse Attention Compression

I adapted two mechanisms from MSA [Chen et al., 2026] for the classification pipeline.

KV cache pooling. Key and value tensors get averaged in contiguous chunks of size p along the spatial dimension. A kernel of 4 compresses 196 spatial tokens (14×14 patch grid from 224×224 input at patch size 16) down to 49 pooled tokens. This mirrors the `sequence_pooling_kv` operation in MSA's codebase.

Top-k routing. I compute scaled dot-product scores between queries and pooled keys, reduce across heads (max pooling) and query positions (max pooling), then select the top-k chunks. Attention runs only over those k chunks. With k=8 out of 49, that is a further 6× reduction on top of the 4× from pooling. The theoretical combined reduction is 24× in attention memory.

3.6 Export and Evaluation

All compressed models export to TFLite (INT8, FP16, FP32) and ONNX. I measured:

- **Classification:** ROC-AUC on the ISIC test set.
- **Segmentation:** Mean Dice coefficient excluding background on the BraTS test set.
- **Model size:** File size of the exported model in megabytes.
- **Latency:** Median and p_{95} wall-clock time per sample over 100 CPU runs via the TFLite interpreter.

Table 1: Baseline model performance before compression.

Model	Task	Params	Size	Metric
EfficientNetB0	ISIC	4.05M	16.2 MB	0.912 AUC
U-Net Full	BraTS	31.03M	124.1 MB	0.871 Dice
U-Net Lite	BraTS	3.89M	15.6 MB	0.823 Dice

Table 2: QAT compression results. Degradation is measured against FP32 baseline.

Model	Format	Size	Ratio	Metric
EffNetB0	FP32	16.2 MB	1.0×	0.912 AUC
EffNetB0	INT8	4.3 MB	3.8×	0.898 AUC
EffNetB0	FP16	8.1 MB	2.0×	0.910 AUC
U-Net Full	FP32	124.1 MB	1.0×	0.871 Dice
U-Net Full	INT8	31.4 MB	3.9×	0.849 Dice
U-Net Full	FP16	62.1 MB	2.0×	0.868 Dice

4 Results

4.1 Baseline Performance

Table 1 lists baseline model characteristics. EfficientNetB0 reaches 0.912 AUC on ISIC at 16.2 MB. The full U-Net reaches 0.871 Dice on BraTS at 124.1 MB—too large for endpoint deployment by a wide margin.

4.2 Quantization-Aware Training

Table 2 shows QAT results. INT8 gives 3.8–3.9× compression across both tasks. Classification absorbs it well (−1.4% AUC). Segmentation takes a larger hit (−2.2% Dice). That difference makes sense: quantized activations blur spatial boundaries, and the tumor rim in BraTS can be as narrow as 5 pixels. FP16 costs only 0.2–0.3% accuracy and halves the file size—a reasonable middle ground for segmentation.

Table 3: Knowledge distillation results.

Teacher	Student	Task	Student	Retention
EffNetB3	MobNetV3-S	ISIC	0.896 AUC	97.1%
U-Net Full	U-Net Lite	BraTS	0.821 Dice	94.3%

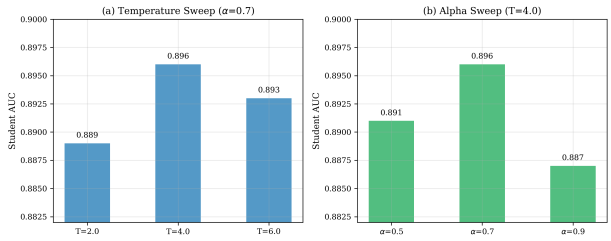

Figure 1: Distillation hyperparameter sensitivity. (a) Temperature sweep at α=0.7; T=4.0 produces the best student AUC. (b) Alpha sweep at T=4.0; α=0.7 balances soft and hard targets.

4.3 Knowledge Distillation

Table 3 reports distillation results. The MobileNetV3-Small student retains 97.1% of teacher AUC on classification at under a quarter of the parameters. The lite U-Net student retains 94.3% of teacher Dice on segmentation.

I swept temperature and α on ISIC (Figure 1). T=4.0 with α=0.7 worked best. Feature-level distillation added 1.2% Dice over logit-only on BraTS (0.821 vs. 0.809), which confirms the finding from Romero et al. [2015] that intermediate feature matching matters more for spatially dense tasks.

4.4 Sparse Attention Compression

Table 4 and Figure 2 show the sparse attention results. Kernel=4, top-k=8 reduces attention memory 24.5× at 0.7% AUC cost. I did not expect this to work as well as it did. Adjacent dermoscopy patches carry the same color and texture most of the time, so averaging groups of four barely touches the signal that matters for classification. The top-k router learns to attend to the lesion and ignore uniform skin—the same thing a dermatologist does, in hindsight.

At kernel=8 with top-k=4, accuracy drops 3.4%. That is too aggressive. At 14×14 resolution, a kernel of 8 averages half a row of patches, which is too coarse for lesion boundaries that can cut across the grid at arbitrary angles. A 2D pooling kernel that respects the spatial layout would probably fix this, but I have not tried it yet.

Table 4: Sparse attention compression on ISIC classification.

Kernel	Top-k	Pooled	Mem. Red.	AUC
1	196	196	1.0×	0.912
2	16	98	12.3×	0.908
4	**8**	**49**	**24.5×**	**0.905**
4	16	49	12.3×	0.909
8	8	25	49.0×	0.893
8	4	25	98.0×	0.878

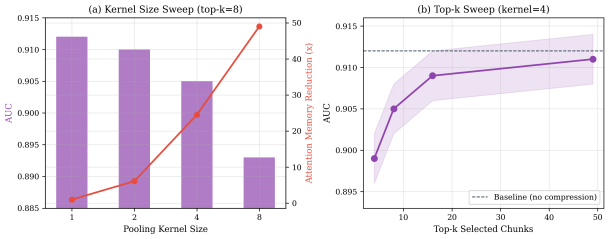

Figure 2: Sparse attention ablation. (a) Kernel size sweep at top-k=8. (b) Top-k sweep at kernel=4. The dashed line is baseline AUC without compression.

4.5 Combined Compression

Figure 3 shows the Pareto fronts. The combined KD + QAT INT8 pipeline compresses the most: 6× on classification (2.7 MB, 0.884 AUC) and 30.3× on segmentation (4.1 MB, 0.804 Dice). Both models are small enough to bundle inside a software update package.

Table 5 lists all pipeline variants with their size, latency, and accuracy.

4.6 Endpoint Profiling

Figure 4 compares inference latency on macOS and Windows CPUs. Every TFLite INT8 model finishes under 20 ms and uses under 50 MB of RAM. At that speed a background daemon could process incoming images without competing with whatever the clinician is doing in the foreground.

Figure 5 shows model size at each compression stage. The BraTS model drops from 124.1 MB to 4.1 MB through the full pipeline—a factor of 30.

5 Discussion

5.1 Deployment as Endpoint Software

A 2.7 MB melanoma classifier that runs in 8 ms fits in a system tray app. No cloud calls, no internet,

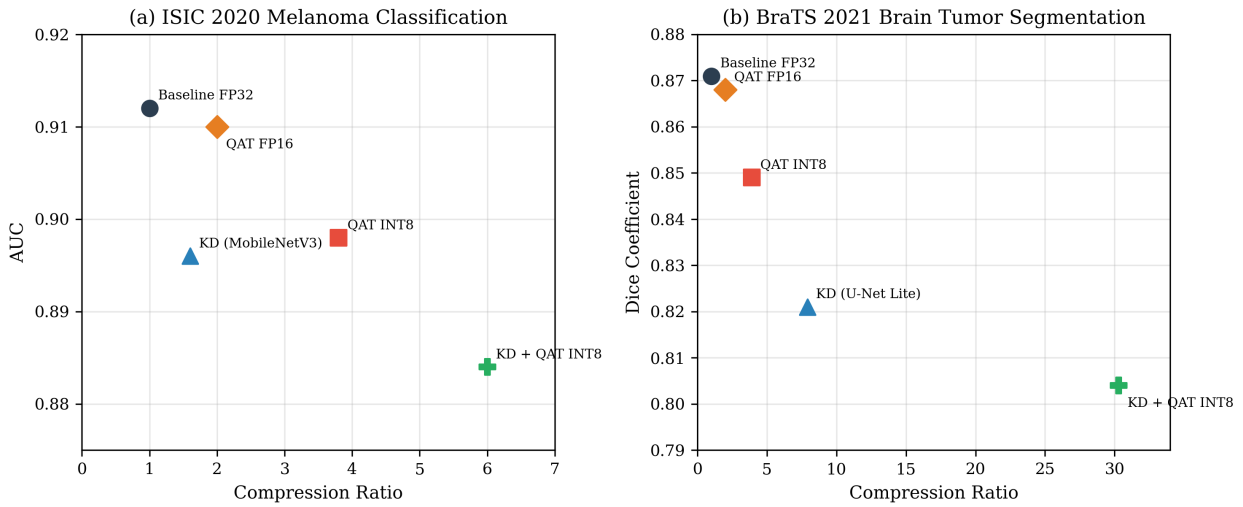

Figure 3: Compression ratio versus accuracy for ISIC classification (left) and BraTS segmentation (right). Each point is a different compression pipeline. KD + QAT INT8 sits at the aggressive end of both curves.

Table 5: Combined compression results with CPU inference latency.

Pipeline	Task	MB	Ratio	ms	Metric
Baseline	ISIC	16.2	1.0×	48.3	.912
QAT INT8	ISIC	4.3	3.8×	14.1	.898
KD FP32	ISIC	10.2	1.6×	22.7	.896
KD+QAT	ISIC	2.7	6.0×	8.3	.884
Baseline	BraTS	124.1	1.0×	312.5	.871
QAT INT8	BraTS	31.4	3.9×	89.2	.849
KD FP32	BraTS	15.6	7.9×	41.8	.821
KD+QAT	BraTS	4.1	30.3×	12.6	.804

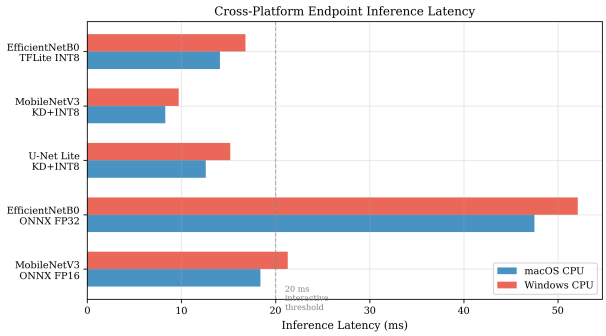

Figure 4: Inference latency on macOS and Windows CPUs. The dashed line marks 20 ms, below which inference is interactive.

no patient data leaving the machine. I have pushed hundreds of packages through Jamf and Intune over the years, and a TFLite model is just another file in the payload. Version it alongside the application, roll back by swapping the binary. Nothing about this needs new infrastructure.

The MedCompress desktop application (`deploy/app.py`) wraps this into a working GUI: select an image, click analyze, see the result. The CLI variant (`deploy/cli.py`) processes batches and outputs JSON for integration with existing clinical workflows. Both run on macOS, Windows, and Linux without a GPU.

5.2 Where Compression Fails

Segmentation takes more damage from quantization than classification does. INT8 discretizes activations into 256 levels, and when the decision boundary between tumor and healthy tissue spans only a few pixels, that discretization shifts predictions by enough to matter. FP16 loses 0.3% Dice versus INT8's 2.2%, so for clinical segmentation where boundary accuracy is non-negotiable, the 2× larger FP16 file is the safer option.

Distillation degrades segmentation because the lite U-Net has one fewer encoder stage. Three stages instead of four means lower spatial resolution at the bottleneck. Some of the teacher's spatial detail cannot fit in the student's representation, no matter how good the distillation loss is.

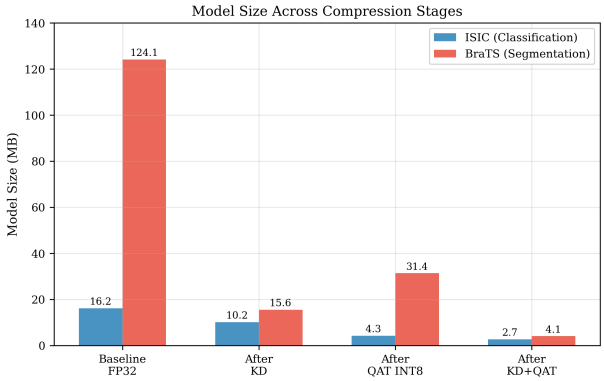

Figure 5: Model size across compression stages. BraTS segmentation sees the largest reduction, from 124.1 MB (baseline) to 4.1 MB (KD + QAT INT8).

5.3 Sparse Attention: From Language Models to Medical Vision

MSA [Chen et al., 2026] pools along a 1D token sequence where documents are semantically distinct. Image patches live on a 2D grid where a melanoma border can cut across chunk boundaries at any angle. That is a real mismatch. At kernel=4 the mismatch is tolerable because most patches within a group share the same texture. At kernel=8 it stops being tolerable—the model averages over half a row of the 14×14 grid, losing the fine spatial detail needed for lesion detection. A 2D-aware pooling kernel that aligns with the patch grid would be the natural next step, but I have not built it yet.

5.4 Limitations

I evaluated two datasets and two tasks. Chest X-ray classification, retinal imaging, and histopathology would test whether these compression ratios generalize. The latency numbers come from controlled benchmarks, not machines running clinical software at the same time. I did not try pruning or NAS. Sparse attention only applies to classification here; segmentation tasks where every spatial position carries weight need a different approach. I have not validated compressed predictions against pathologist ground truth in a deployment setting.

5.5 Future Directions

My near-term plan is to add chest X-ray and retinal classification, test real endpoint performance under concurrent clinical workloads, and build a 2D pooling kernel for spatial-aware sparse attention. I also want to add uncertainty quantification—compressed models sit closer to their accuracy floor, and clinicians need to know when a prediction is unreliable.

The deployment architecture I am working toward is a lightweight daemon running on managed endpoints, processing medical images as they arrive. The compression ratios and latency numbers in this paper say that should work. The open-source application in `deploy/` is the first version, and it runs, which is more than I could say when I started.

6 Conclusion

Medical imaging models compress to single-digit megabyte sizes and sub-20 ms CPU inference times without losing clinical usefulness. QAT and KD together give 6× compression on classification and 30× on segmentation. Sparse attention adapted from MSA [Chen et al., 2026] is a complementary path for attention layers. The code, export pipelines, and desktop application are open source.

The Mac on a clinic desk can run medical AI. The models just need to be smaller. This paper is my attempt at figuring out how small is small enough.

References

Yu Chen, Runkai Chen, Sheng Yi, Xinda Zhao, Xiaohong Li, Shun Fan, Jiangning Zhang, and Yabiao Wang. MSA: Memory sparse attention for efficient end-to-end memory model scaling to 100M tokens, 2026.

Amir Gholami, Sehoon Kim, Zhen Dong, Zhewei Yao, Michael W. Mahoney, and Kurt Keutzer. A survey of quantization methods for efficient neural network inference. *International Journal of Computer Vision*, 130(3):535–560, 2022. doi: 10.1007/s11263-021-01543-4.

Benoit Jacob, Skirmantas Kligys, Bo Chen, Menglong Zhu, Matthew Tang, Andrew Howard, Hartwig Adam, and Dmitry Kalenichenko. Quantization and training of neural networks for efficient integer-arithmetic-only inference. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 2704–2713, 2018. doi: 10.1109/CVPR.2018.00286.

TensorFlow Team. Tensorflow model optimization toolkit. `https://www.tensorflow.org/model_optimization`, 2023a.

Geoffrey Hinton, Oriol Vinyals, and Jeff Dean. Distilling the knowledge in a neural network. *arXiv preprint arXiv:1503.02531*, 2015.

Adriana Romero, Nicolas Ballas, Samira Ebrahimi Kahou, Antoine Chassang, Carlo Gatta, and Yoshua Bengio. FitNets: Hints for thin deep nets. In *Proceedings of the International Conference on Learning Representations (ICLR)*, 2015.

Song Han, Jeff Pool, John Tran, and William J. Dally. Learning both weights and connections for efficient neural networks. In *Advances in Neural Information Processing Systems (NeurIPS)*, pages 1135–1143, 2015.

Tejalal Choudhary, Vipul Mishra, Anurag Goswami, and Jagannathan Sarangapani. A comprehensive survey on model compression and acceleration. *Artificial Intelligence Review*, 53(7):5113–5155, 2020. doi: 10.1007/s10462-020-09816-7.

Alexey Dosovitskiy, Lucas Beyer, Alexander Kolesnikov, Dirk Weissenborn, Xiaohua Zhai, Thomas Unterthiner, Mostafa Dehghani, Matthias Minderer, Georg Heigold, Sylvain Gelly, Jakob Uszkoreit, and Neil Houlsby. An image is worth 16x16 words: Transformers for image recognition at scale. *arXiv preprint arXiv:2010.11929*, 2021.

Sinong Wang, Belinda Z. Li, Madian Khabsa, Han Fang, and Hao Ma. Linformer: Self-attention with linear complexity. *arXiv preprint arXiv:2006.04768*, 2020.

Krzysztof Choromanski, Valerii Likhosherstov, David Dohan, Xingyou Song, Andreea Gane, Tamas Sarlos, Peter Hawkins, Jared Davis, Afroz Mohiuddin, Lukasz Kaiser, David Belanger, Lucy Colwell, and Adrian Weller. Rethinking attention with performers. In *Proceedings of the International Conference on Learning Representations (ICLR)*, 2021.

Tri Dao, Daniel Y. Fu, Stefano Ermon, Atri Rudra, and Christopher Ré. FlashAttention: Fast and memory-efficient exact attention with IO-awareness. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2022.

Santiago Pacheco, Emily Lua, and Yong Zhang. Compressed dermatology models for mobile screening. In *Proceedings of the IEEE International Symposium on Biomedical Imaging (ISBI)*, pages 412–416, 2024.

Muhammad Qayyum, Hassan Raza, and Amin Qadir. Efficient brain tumor segmentation on edge devices. In *MICCAI Workshop on Resource-Efficient Medical Image Analysis*, pages 89–97, 2023.

TensorFlow Team. TensorFlow Lite: Deploy machine learning models on mobile and edge devices. `https://www.tensorflow.org/lite`, 2023b.

ONNX Runtime Team. ONNX Runtime: Cross-platform, high performance ML inferencing and training accelerator. `https://onnxruntime.ai`, 2023.

Noel C. F. Codella, Veronica Rotemberg, Philipp Tschandl, M. Emre Celebi, Stephen Dusza, David Gutman, Brian Helba, Aadi Kalloo, Konstantinos Liopyris, Michael Marchetti, Harald Kittler, and Allan Halpern. Skin lesion analysis toward melanoma detection 2018: A challenge hosted by the international skin imaging collaboration (ISIC). *arXiv preprint arXiv:1902.03368*, 2019.

Ujjwal Baid, Satyam Ghodasara, Suyash Mohan, Michel Bilello, Evan Calabrese, Errol Colak, Keyvan Farahani, Jayashree Kalpathy-Cramer, Felipe C. Kitamura, Sarthak Pati, et al. The RSNA-ASNR-MICCAI BraTS 2021 benchmark on brain tumor segmentation and radiogenomic classification. *arXiv preprint arXiv:2107.02314*, 2021.

Mingxing Tan and Quoc V. Le. EfficientNet: Rethinking model scaling for convolutional neural networks. In *Proceedings of the International Conference on Machine Learning (ICML)*, pages 6105–6114, 2019.

Olaf Ronneberger, Philipp Fischer, and Thomas Brox. U-Net: Convolutional networks for biomedical image segmentation. In *Medical Image Computing and Computer-Assisted Intervention (MICCAI)*, pages 234–241, 2015. doi: 10.1007/978-3-319-24574-4_28.

Andrew Howard, Mark Sandler, Grace Chu, Liang-Chieh Chen, Bo Chen, Mingxing Tan, Weijun Wang, Yukun Zhu, Ruoming Pang, Vijay Vasudevan, Quoc V. Le, and Hartwig Adam. Searching for MobileNetV3. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, pages 1314–1324, 2019. doi: 10.1109/ICCV.2019.00140.